Bank Transaction Analytics and Fraud Detection

Big Data Analytics — Project Report and Technical Explanation

Python · Apache Spark (Spark SQL, MLlib) · MongoDB · Machine Learning

Submitted by: _____

Roll number: _____

Course / Instructor: _____

Repository: https://github.com/JayQuery-007/bank_transaction_analysis

October 2026

Contents

1. Summary 4
2. Introduction 5
2.1 Problem statement 5
2.2 Objectives 5
2.3 Why this is a big data problem 5
3. Dataset 6
3.1 Source 6
3.2 Columns 6
3.3 Data quality check results 6
4. Technology stack 8
5. System architecture 9
6. Pipeline implementation 10
6.1 Shared Spark setup (spark_jobs/common.py) 10
6.2 Step 1 — Ingest (spark_jobs/ingest.py) 10
6.3 Step 2 — Analytics with Spark SQL (spark_jobs/analytics.py) 11
6.4 Step 3 — Fraud detection model (spark_jobs/train_model.py) 11
6.5 MongoDB serving layer 13
6.6 Backend API (backend/app) 13
6.7 Dashboard (frontend/) 13
7. How a transaction is classified as fraud or legitimate 15
7.1 Where the fraud labels come from 15
7.2 What the model looks at 15
7.3 How the Random Forest makes a decision 15
7.4 Worked examples from the running application 16
7.5 How to explain it to your professor 16
8. Application of Big Data in this project 17
8.1 The five Vs of Big Data 17
8.2 Big data techniques used 17
8.3 Why not just Excel or pandas? 17
8.4 Real-world uses of this kind of system 18
9. Results and findings 19
9.1 Which transaction types are affected? 19
9.2 When does fraud happen? 20
9.3 What do fraudulent transactions look like? 20
9.4 Model results 21
10. Running the project 24
10.1 Requirements 24
10.2 Start the dashboard 24

10.3 Rebuild everything from the raw data 24
10.4 Verified run 24
10.5 Dashboard screenshots 25
11. Technical challenges and how they were solved 28
11.1 Spark cannot write files on Windows without winutils.exe 28
11.2 PySpark Python workers crash with Python 3.13 on Windows 28
11.3 Long file names in the saved model 28
11.4 Making the repository runnable without the 494 MB dataset 28
12. Limitations and future work 29
12.1 Limitations 29
12.2 Future work 29
13. Viva preparation: likely questions 30
14. Project structure 32
References 32

1. Summary

This project builds a batch big data pipeline for a dataset of 6,362,620 mobile-money transactions (494 MB). Apache Spark loads the raw CSV into a Parquet data lake, Spark SQL is used to analyse where, when and how fraud happens, and Spark MLlib trains a model that detects fraudulent transactions. The results are stored in MongoDB and shown on a web dashboard served by a FastAPI backend.

Main results:

- Fraud occurs **only in TRANSFER and CASH_OUT** transactions (0.77% and 0.18% of those types). The other three types contain no fraud.
- The provider's existing rule (`isFlaggedFraud`) caught only **16 of 8,197** frauds (0.2%).
- Fraud volume is almost constant through the day, so when legitimate activity drops at night the fraud rate rises to **22.3% at 5:00**.
- **97.7%** of frauds empty the sender's account, compared with 42.7% of legitimate transfers and cash-outs.
- The Random Forest model caught **2,401 of 2,407** frauds in the unseen test set with **17 false alarms** (precision 0.993, recall 0.998).

2. Introduction

2.1 Problem statement

Banks and payment providers process millions of transactions every day. Only a tiny fraction are fraudulent, but each one is a direct financial loss and a trust problem for the customer. Rule-based checks (for example "flag transfers above a fixed amount") are simple to run but miss most fraud. The data is too large to inspect by hand, so it needs distributed processing for both analysis and detection.

2.2 Objectives

1. Load a large raw transaction log into an efficient, query-friendly storage format.
2. Check data quality and prepare the data (cleaning, derived features).
3. Describe the data: which transaction types, times, amounts and accounts are involved in fraud.
4. Train and evaluate a fraud detection model and compare it with the existing rule.
5. Store the results in a database and present them through an API and a dashboard.

2.3 Why this is a big data problem

- **Volume:** 6,362,620 rows and 494 MB for one month of a single provider. A real bank produces this volume every few days, so the solution has to scale beyond a single machine's memory.
- **Scalability:** All processing is written with Spark DataFrames and Spark SQL. The same code runs on a laptop (local mode) or on a cluster with the data in HDFS or cloud storage, without changes.
- **Variety of workloads:** The same data is used for batch ETL, SQL analytics and distributed machine learning, which is what the Spark ecosystem is designed for.

3. Dataset

3.1 Source

PaySim — Synthetic Financial Datasets For Fraud Detection (Kaggle, dataset id `ealaxi/paysim1`).

Real bank transactions cannot be published because of customer privacy and banking regulations. PaySim was generated by an agent-based simulator that was calibrated on one month of real transaction logs from a mobile money service operating in an African country. It reproduces the statistical behaviour of the real data, including fraud, without exposing any real customers. It is the standard public benchmark for transaction fraud detection and is widely used in research and teaching.

> **Important for presentation:** PaySim is simulated from real logs. It is real in the sense that it is a published, widely used Kaggle dataset, not something generated for this project, but it is not raw bank data. Section 12 explains how this affects the results.

3.2 Columns

Column	Type	Description
`step`	integer	Hour of the simulation, 1 to 744 (31 days)
`type`	string	CASH_IN, CASH_OUT, DEBIT, PAYMENT or TRANSFER
`amount`	double	Transaction amount in local currency
`nameOrig`	string	Sender (customer) account ID
`oldbalanceOrg`	double	Sender's balance before the transaction
`newbalanceOrig`	double	Sender's balance after the transaction
`nameDest`	string	Receiver account ID (IDs starting with M are merchants)
`oldbalanceDest`	double	Receiver's balance before the transaction
`newbalanceDest`	double	Receiver's balance after the transaction
`isFraud`	integer	1 if the transaction is fraudulent (the label)
`isFlaggedFraud`	integer	1 if flagged by the provider's existing rule (attempts to transfer more than 200,000 at once)

3.3 Data quality check results

These checks are run by the ingest job on the full raw file:

Check	Result
Total rows	6,362,620
Rows with null values	0
Duplicate rows	0
Negative amounts	0
Zero amounts (removed)	16 (all CASH_OUT, all labelled fraud)
Distinct sender accounts	6,353,307
Distinct receiver accounts	2,722,362
Rows written to the data lake	6,362,604

The data is clean. The 16 zero-amount rows are not valid transactions, so they are removed, leaving 8,197 fraud cases. Almost every sender appears only once (6,353,307 distinct senders in 6,362,620 rows), which limits per-customer behavioural analysis.

4. Technology stack

The course allows several technology combinations. This project uses **Python + Spark + MongoDB + Machine Learning**, with Parquet as the storage format and FastAPI for the dashboard.

Technology	Role in the project	Why it was chosen
Python 3	Language for all jobs, the API and the notebook	Main language of the course; PySpark gives full access to Spark
Apache Spark 3.5 (PySpark)	Ingest, cleaning, feature engineering	Distributed, lazy DataFrame engine that scales from a laptop to a cluster
Spark SQL	All descriptive analytics (7 queries)	Standard SQL over large data, optimised by Spark's Catalyst optimiser
Spark MLlib	Logistic Regression and Random Forest training and evaluation	Trains on the full 2.77M-row training data in parallel, with no sampling needed
Apache Parquet	Data lake storage format	Columnar and compressed, stores the schema, supports partition pruning
MongoDB 8	Serving database for analysis results	JSON documents match what the API and dashboard need; flexible schema per result set
FastAPI + Uvicorn	REST API and web server for the dashboard	Lightweight, fast and auto-documents its endpoints
HTML, CSS, Chart.js	Dashboard	Simple static front end with no build step; Chart.js is bundled so it works offline
Jupyter, matplotlib	Exploratory data analysis notebook	Interactive exploration and plots
kagglehub	Dataset download	Official Kaggle client
Git / GitHub	Version control and submission	Code, results and trained model are versioned together

5. System architecture

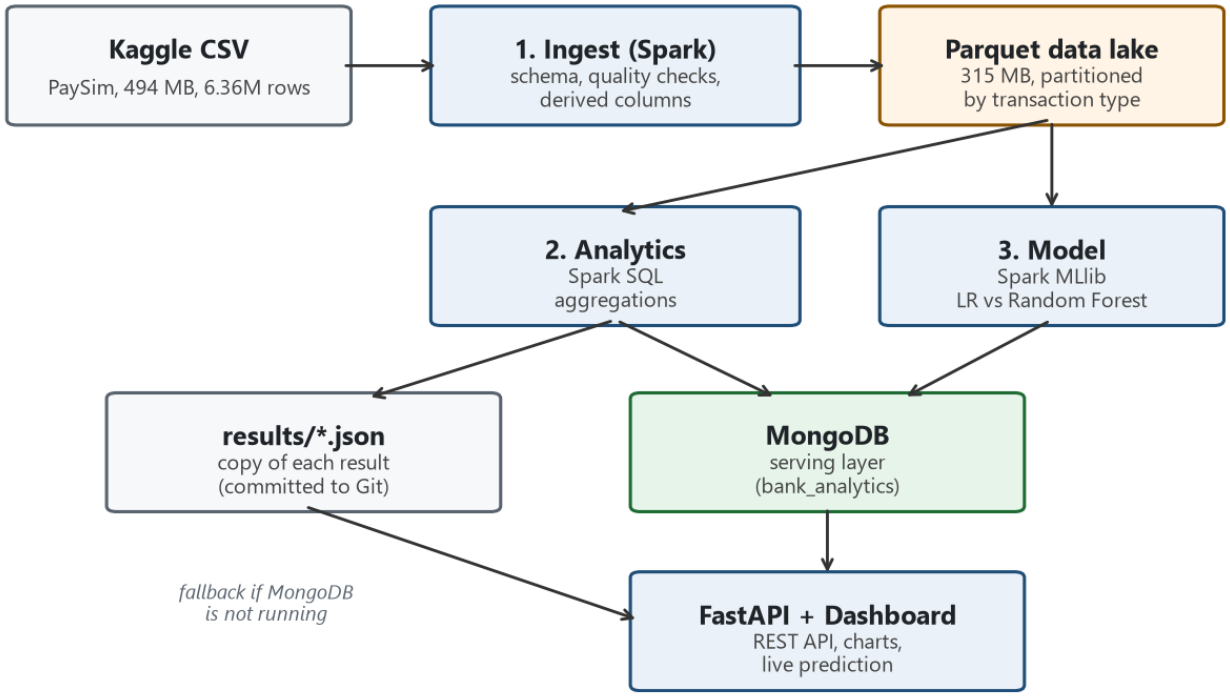


Figure 1. Data flow through the pipeline

The architecture follows the usual split between a batch layer and a serving layer:

- **Batch layer (Spark + Parquet):** the full 6,362,604 rows live in the Parquet data lake. All heavy processing (cleaning, aggregations, model training) runs here.
- **Serving layer (MongoDB):** only the results the dashboard needs are stored: aggregates, model metrics and flagged transactions. These are a few thousand documents, not millions, so queries are instant.
- **Presentation layer (FastAPI + dashboard):** reads from MongoDB and renders charts and tables. It also loads the trained Spark model to score single transactions on request.

Every result set is written both to MongoDB and to a JSON file in `results/`. The JSON copies are committed to Git, so anyone who clones the repository can open the dashboard immediately, without downloading the data or installing MongoDB. When MongoDB is running, the API reads from it.

6. Pipeline implementation

The pipeline has four steps. `python run_pipeline.py` runs them in order. Each step is a separate Python module that can also be run on its own.

Step	Script	Input → Output	Runtime*
0. Download	`data/download_data.py`	Kaggle → data/raw/paysim.csv	~20 s
1. Ingest	`spark_jobs/ingest.py`	CSV → Parquet lake	~20–40 s
2. Analytics	`spark_jobs/analytics.py`	Parquet → 7 result sets in MongoDB	~25–40 s
3. Model	`spark_jobs/train_model.py`	Parquet → model, metrics, flagged transactions	~2.5 min

**Measured on a 16 GB Windows laptop with Spark in local mode using all CPU cores.*

6.1 Shared Spark setup (spark_jobs/common.py)

All jobs get their Spark session from `get_spark()`, which:

- runs Spark in local mode on all cores (local[*]) with 4 GB of driver memory,
- enables Apache Arrow for fast transfer of results from Spark to pandas,
- finds Java automatically (JAVA_HOME, the PATH, or a JDK bundled in the project folder),
- on Windows only, plugs in a small custom file system class (explained in Section 11).

The same file has `publish(name, records)`, which writes a result set to MongoDB (one collection per result) and to `results/<name>.json`.

6.2 Step 1 — Ingest (spark_jobs/ingest.py)

Explicit schema

The CSV is read with a hand-written `StructType` schema instead of `inferSchema`. Schema inference would need an extra full pass over all 6.36M rows just to guess the types, and it can guess wrong. With an explicit schema, Spark reads the file once with the correct types.

Cleaning and renaming

Columns are renamed to consistent snake_case (for example oldbalanceOrg → old_balance_orig, which also fixes the inconsistent 'Org'/'Orig' naming in the source). Zero-amount rows are removed.

Derived columns

Column	Formula	Purpose
`day`	(step − 1) / 24 + 1	Day of the month (1–31)
`hour_of_day`	step mod 24	Time-of-day analysis and model feature
`dest_is_merchant`	name_dest starts with 'M'	Merchants have no balance information
`error_balance_orig`	new_balance_orig + amount − old_balance_orig	How far the sender's recorded balance is from what the amount implies
`error_balance_dest`	old_balance_dest + amount − new_balance_dest	Same check for the receiver
`orig_emptied`	old_balance_orig > 0 and new_balance_orig = 0	Did the transaction empty the sender's account?

Parquet data lake, partitioned by type

The cleaned data is written to `data/lake/transactions/` in Parquet format, partitioned by transaction type, so each type gets its own folder (`type=TRANSFER/`, `type=CASH_OUT/` and so on).

- **Smaller:** 494 MB of CSV becomes 315 MB of Parquet, even after adding six derived columns, thanks to columnar storage and Snappy compression.
- **Faster:** queries read only the columns they use, and the schema is stored with the data, so there is no parsing step.
- **Partition pruning:** the model only needs TRANSFER and CASH_OUT. Because of the partitioning, Spark reads only those two folders (2.77M rows) and skips the other 3.6M rows entirely. The notebook shows this in Spark's physical plan as a `PartitionFilters` entry.

6.3 Step 2 — Analytics with Spark SQL (spark_jobs/analytics.py)

The Parquet lake is registered as a temporary SQL view called `tx` and cached in memory. Seven SQL queries are then run, and each result is published to its own MongoDB collection:

Result set (collection)	Question it answers
`overview`	Totals: transactions, value, fraud count and value, fraud rate, accounts, days
`type_summary`	Which transaction types carry fraud, and at what rate?
`hourly`	How do volume and fraud rate change by hour of day?
`daily`	Volume and fraud per day across the 31 days
`amount_bands`	How are fraud and legitimate amounts distributed (order-of-magnitude bands)?
`balance_patterns`	How do account balances behave in fraud vs legitimate transactions?
`repeat_fraud_destinations`	Which receiver accounts got more than one fraudulent transaction (possible mule accounts)?

Example: the hourly query.

```
SELECT hour_of_day,
       COUNT(*)                       AS transactions,
       SUM(is_fraud)                  AS fraud_transactions,
       ROUND(100 * AVG(is_fraud), 3)  AS fraud_rate_pct
FROM tx
GROUP BY hour_of_day
ORDER BY hour_of_day
```

6.4 Step 3 — Fraud detection model (spark_jobs/train_model.py)

Training data

The analytics showed that fraud only occurs in TRANSFER and CASH_OUT, so the model is trained on those 2,770,393 transactions only. The data is split at random into 70% training (1,938,956 rows) and 30% test (831,437 rows, 2,407 frauds), with a fixed seed so the results are reproducible. The test set is never used for training. It plays the role of new, unseen transactions.

Features

Feature	Meaning
`is_transfer`	1 for TRANSFER, 0 for CASH_OUT

Feature	Meaning
`amount`	Transaction amount
`old_balance_orig`, `new_balance_orig`	Sender's balance before and after
`old_balance_dest`, `new_balance_dest`	Receiver's balance before and after
`error_balance_orig`, `error_balance_dest`	Balance inconsistencies (Section 6.2)
`hour_of_day`	Hour 0–23

Class imbalance

Fraud is only 0.30% of the training data, roughly 1 fraud for every 337 legitimate transactions. A model can reach 99.7% accuracy by never predicting fraud, which is useless. Two measures address this:

- Logistic Regression is trained with **class weights**: each fraud row counts 337 times, so both classes carry equal total weight.
- Models are judged on **precision, recall, F1 and AUC-PR**, not accuracy.

Models compared

- **Logistic Regression:** a linear model. Features are standardised with StandardScaler first. It is fast and easy to interpret, but it can only learn linear boundaries.
- **Random Forest:** an ensemble of 60 decision trees (maximum depth 12). Each tree learns rules such as "the sender's balance became 0 and the amount equals the old balance", and the trees vote. It can capture non-linear patterns and interactions between features.
- **Existing rule (baseline):** the dataset's own `isFlaggedFraud` column, evaluated on the same test set. It represents the system already in place.

Each model is built as a Spark ML Pipeline: VectorAssembler (combines the feature columns into one vector), then an optional StandardScaler, then the classifier.

Evaluation metrics

Metric	Definition	Meaning here
Precision	TP / (TP + FP)	Of the transactions flagged, how many are really fraud (few false alarms)
Recall	TP / (TP + FN)	Of all frauds, how many were caught
F1	2·P·R / (P + R)	Single score balancing precision and recall
AUC-ROC	Area under the ROC curve	Ranking quality across all thresholds
AUC-PR	Area under the precision-recall curve	Like AUC-ROC but focused on the rare class, so better suited to imbalanced data
Confusion matrix	TP, FP, FN, TN counts	Exact numbers of correct and wrong decisions

TP = true positive (fraud correctly flagged), FP = false positive (legitimate transaction flagged), FN = false negative (fraud missed), TN = true negative.

Outputs

- metrics for all three models (`model_metrics`), training details (`model_info`)
- Random Forest feature importance (`feature_importance`)

- precision and recall at alert thresholds from 0.1 to 0.9 (`threshold_curve`)
- every test transaction that was flagged or was a missed fraud, 2,424 rows with probability and outcome (`flagged_transactions`)
- the trained Random Forest pipeline, saved in Spark ML format to `models/fraud_rf/` (347 KB)

6.5 MongoDB serving layer

Database `bank_analytics` holds one collection per result set, 13 in total. Each run of the pipeline replaces a collection's contents, so the database always matches the latest run.

Collection	Documents	Written by
`data_quality`	1	ingest
`overview`	1	analytics
`type_summary`	5	analytics
`hourly`	24	analytics
`daily`	31	analytics
`amount_bands`	7	analytics
`balance_patterns`	2	analytics
`repeat_fraud_destinations`	43	analytics
`model_metrics`	3	train_model
`model_info`	1	train_model
`feature_importance`	9	train_model
`threshold_curve`	9	train_model
`flagged_transactions`	2,424	train_model

6.6 Backend API (backend/app)

Endpoint	Description
`GET /api/source`	Shows whether data is coming from MongoDB or results/*.json
`GET /api/results/{name}`	Returns any result set, for example /api/results/hourly
`GET /api/flagged`	Paged flagged transactions, optionally filtered by outcome (true positive, false positive, missed fraud)
`POST /api/predict`	Scores one transaction with the saved Random Forest and returns the fraud probability
`GET /`	Serves the dashboard

`store.py` decides where to read from (MongoDB if reachable, otherwise the JSON files). `predict.py` starts a Spark session on the first prediction request (about 5–7 seconds), loads the model once and reuses it, so later predictions take about 0.1 seconds. Input is validated by Pydantic: type must be TRANSFER or CASH_OUT, amounts must be positive, hour must be 0–23.

6.7 Dashboard (frontend/)

A single static page arranged as a report. Each section answers one question:

1. Dataset overview: totals and data quality
2. Which transaction types are affected?
3. When does fraud happen? (hourly volume and fraud rate)
4. What do fraudulent transactions look like? (amounts, balances, mule accounts)
5. Fraud detection model: comparison table, confusion matrix, feature importance, threshold curve
6. Flagged transactions: filterable, paged table
7. Check a transaction: a form that scores a transaction with the model

The text findings on the page are generated from the data, not hard-coded, so they stay correct if the pipeline is rerun.

7. How a transaction is classified as fraud or legitimate

7.1 Where the fraud labels come from

Every transaction in the dataset already carries a label, `isFraud` = 1 (fraud) or 0 (legitimate). The simulator knows which of its simulated customers are fraudsters, so it records the ground truth. In a real bank, labels come from confirmed cases: chargebacks, customer complaints and investigations by the fraud team.

This project does not define fraud with hand-written rules. The model **learns** what fraud looks like from the 8,197 labelled fraud examples. This is called **supervised learning**: learn from past, labelled examples, then apply what was learned to new transactions.

7.2 What the model looks at

For every transaction the model sees nine numbers:

- the type (TRANSFER or CASH_OUT) and the amount,
- the sender's balance before and after the transaction,
- the receiver's balance before and after the transaction,
- two consistency checks: do the balances change by exactly the amount? (error_balance_orig, error_balance_dest),
- the hour of the day.

It does not see names or account numbers. It judges the behaviour of the transaction, not who made it.

7.3 How the Random Forest makes a decision

A decision tree is a flowchart of yes/no questions. During training, Spark picks each question (a feature and a threshold) as the one that best separates fraud from legitimate transactions in the training data. Below is a real path from the first tree of the trained model, written in plain words:

```
Was the sender's balance before the transaction at most 327,397?        yes
  Did it happen at night (hour 7 or earlier)?                           yes
    Is the receiver's balance afterwards at most 55,262?                yes
      Did the sender's balance drop by exactly the amount?              yes
        Did the receiver's balance rise by exactly the amount?          yes
          Is the sender's balance afterwards (almost) 0?
              yes  ->  FRAUD
              no   ->  LEGITIMATE
```

This one path describes the classic account-takeover pattern: at night, the whole balance is moved out and the sender's account is left empty.

A Random Forest is a group of 60 such trees (14,182 decision points in total). Each tree is trained on a random sample of the rows and features, so each learns slightly different rules. To classify a new transaction:

1. Spark computes the nine features for the transaction.
2. The transaction is passed through all 60 trees. Each tree follows its questions down to a leaf, which gives a fraud probability.
3. The forest averages the 60 answers into one fraud probability between 0 and 1.
4. If the probability is **0.5 or higher**, the transaction is classified as **fraud**; otherwise it is **legitimate**.

Using many trees instead of one makes the decision more stable. A single tree can be fooled by one unusual value, but the majority of 60 trees rarely is.

7.4 Worked examples from the running application

Transaction	Key signals	Fraud probability	Decision
CASH_OUT of 2,658,595.89 at 2:00. Sender: 2,658,595.89 → 0. Receiver: 286,931 → 2,945,527	Night time; entire balance taken; sender left at exactly 0	1.000	**Fraud**
CASH_OUT of 18,500 at 14:00. Sender: 42,000 → 23,500. Receiver: 125,000 → 143,500	Daytime; part of the balance; balances add up	0.000	**Legitimate**

The first example is a real fraud from the test set, which the model had never seen during training.

7.5 How to explain it to your professor

> "The dataset contains 8,197 transactions confirmed as fraud. Using Spark MLlib, I trained a Random Forest of 60 decision trees on 1,938,956 labelled transactions. Each tree learned yes/no rules about the amount, the balances before and after, and the time of day. To classify a new transaction, all 60 trees vote, and the share of fraud votes is the fraud probability. Above 0.5, the transaction is flagged as fraud. The model learned that fraud usually empties the sender's whole balance, often at night, and leaves inconsistent balances. On 831,437 transactions it had never seen, it caught 2,401 of 2,407 frauds with 17 false alarms, while the bank's existing rule caught 4."

8. Application of Big Data in this project

8.1 The five Vs of Big Data

V	In this project	In a real bank
Volume	6,362,620 transactions (494 MB) for one month of one provider. A year would be about 76 million rows.	Millions of transactions per day across cards, transfers and mobile payments; years of history kept for analysis and regulation.
Velocity	About 8,552 transactions per simulated hour, processed in batch.	Each payment must be scored in milliseconds before it is approved, which needs streaming (Kafka + Spark Structured Streaming).
Variety	Structured transaction records, plus results stored as flexible JSON documents in MongoDB.	Transactions combined with device data, locations, login logs and customer profiles, often semi-structured.
Veracity	Data-quality checks on every column; 16 invalid rows removed; balance inconsistencies measured and used as features.	Missing, delayed and inconsistent records from many systems must be detected and handled.
Value	The model caught 2,401 of 2,407 test frauds, worth 3.51 bn of 3.52 bn (99.95% of fraud value). The existing rule caught 4.	Prevented losses, fewer false alarms for investigators, protected customers and regulatory compliance.

8.2 Big data techniques used

Technique	Where it is used	Benefit
Distributed processing	All jobs run on Spark, which splits data into partitions processed in parallel on all CPU cores	Same code scales from a laptop to a cluster of machines
Lazy evaluation and query optimisation	Spark builds the whole plan (read → filter → aggregate) and optimises it before running	Avoids unnecessary work and intermediate copies
Data lake in a columnar format	Raw CSV converted to Parquet	494 MB → 315 MB; only needed columns are read
Partitioning and partition pruning	Lake partitioned by transaction type	Model reads 2.77M rows and skips 3.6M without opening them
Schema on write	Explicit schema in the ingest job	No extra full pass over the data to guess types
In-memory caching	Transactions view cached for the 7 analytics queries	Data is read from disk once, not seven times
Distributed machine learning	Spark MLlib trains on all 1.94M training rows; trees are built in parallel	No need to sample the data down to fit in memory
Batch layer and serving layer	Parquet lake for processing, MongoDB for serving results	Heavy work runs offline; the dashboard reads a few thousand small documents instantly
NoSQL database	MongoDB stores each result set as JSON documents	Flexible schema; results map directly to the API responses

8.3 Why not just Excel or pandas?

- **Excel** can hold at most 1,048,576 rows per sheet. This dataset has 6,362,620, six times the limit.

- **pandas** loads all data into one machine's memory. It can manage one month here, but not years of data from a real bank, and it cannot spread the work across machines. Spark processes data in partitions, spills to disk when memory is full, and scales out by adding machines.
- **Spark SQL and MLlib** give SQL analytics and machine learning in the same engine over the same data, with no export or copy between tools.

8.4 Real-world uses of this kind of system

- **Real-time fraud scoring:** card and mobile payments checked before approval.
- **Anti-money-laundering:** finding mule accounts and suspicious chains of transfers, like the 43 repeat receivers found here.
- **Transaction monitoring dashboards:** for fraud and risk teams.
- **Customer risk scoring and regulatory reporting:** over the full transaction history.

9. Results and findings

9.1 Which transaction types are affected?

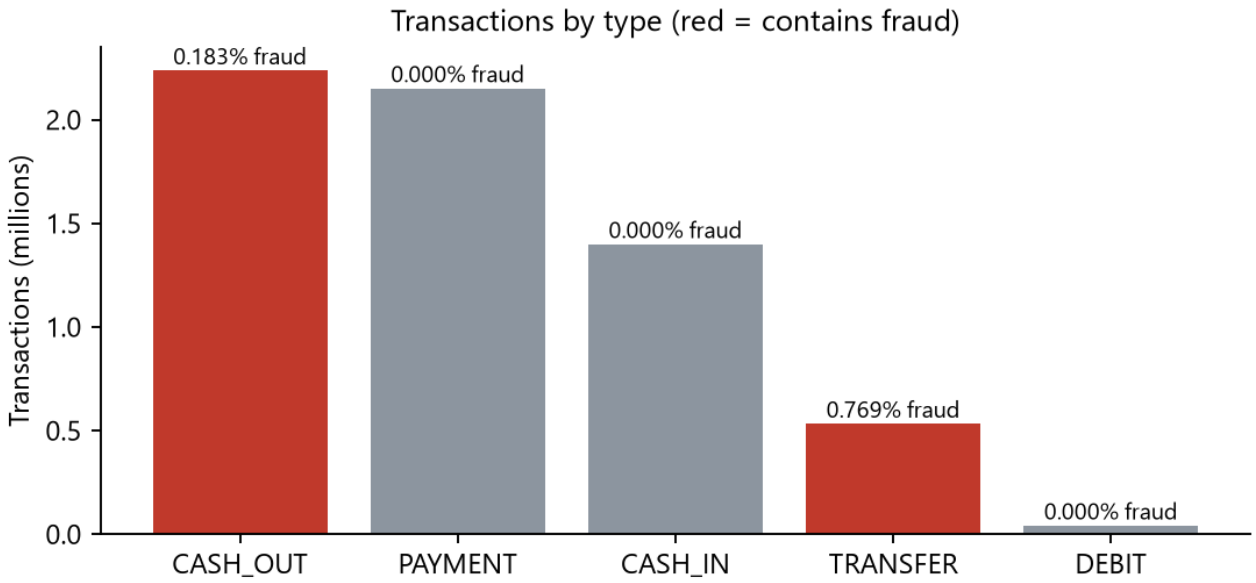


Figure 2. Transactions by type; red bars contain fraud

Type	Transactions	Avg amount	Fraud	Fraud rate
CASH_OUT	**2,237,484**	**176,275**	**4,100**	**0.183%**
PAYMENT	2,151,495	13,058	0	0.000%
CASH_IN	1,399,284	168,920	0	0.000%
TRANSFER	**532,909**	**910,647**	**4,097**	**0.769%**
DEBIT	41,432	5,484	0	0.000%

Fraud exists only in TRANSFER and CASH_OUT. This matches how fraud works in the simulation: a fraudster takes over an account, **transfers** the balance to an account they control, and then **cashes it out**. The near-equal fraud counts in the two types (4,100 and 4,097) support this two-step pattern. The existing rule flagged only 16 transactions in the whole dataset. All of them were fraud, but that is 0.2% of all fraud.

9.2 When does fraud happen?

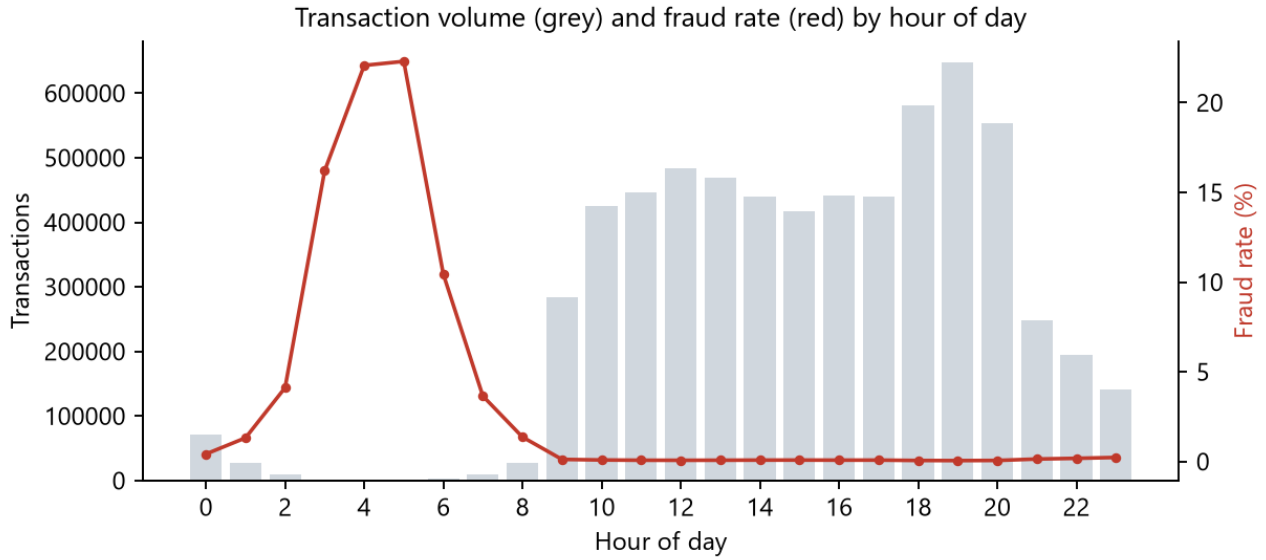


Figure 3. Hourly volume and fraud rate

The number of frauds per hour slot is almost constant (274–371). Legitimate activity follows a daily cycle and nearly stops between 2:00 and 7:00, so in those hours fraud makes up a large share of all transactions, peaking at 22.3% at 5:00. During business hours the fraud rate is below 0.1%. In practice, a transaction at 4 a.m. deserves more scrutiny, so hour_of_day is included as a model feature.

9.3 What do fraudulent transactions look like?

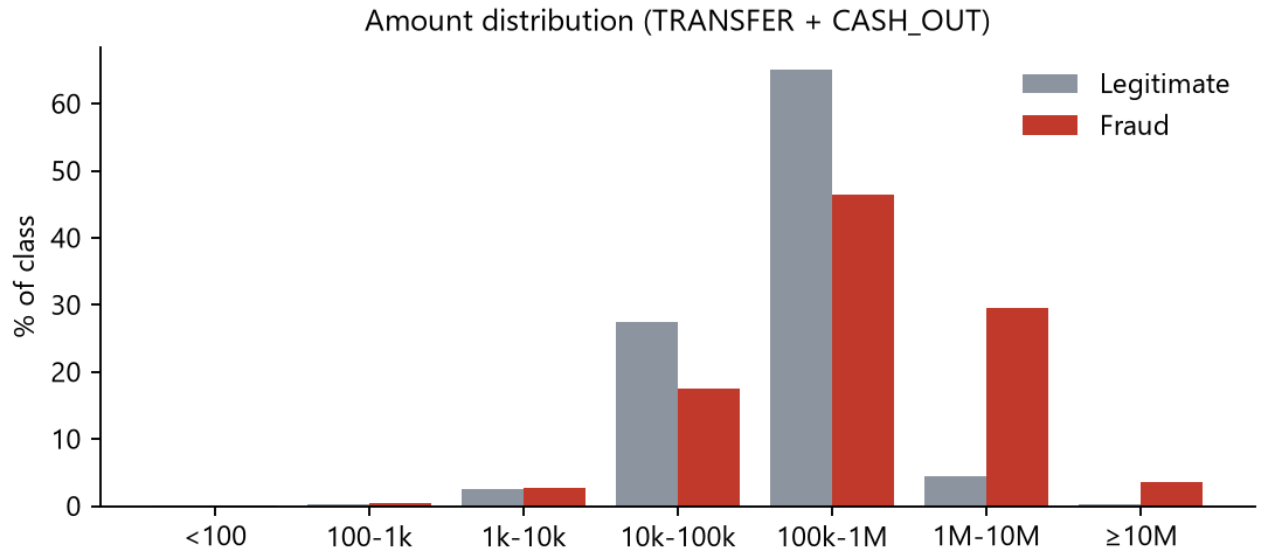


Figure 4. Amount distribution, TRANSFER and CASH_OUT only

Measure (TRANSFER + CASH_OUT)	Legitimate	Fraud
Transactions	2,762,196	8,197
Average amount	314,116	1,470,833
Median amount	171,021	442,290
Sender account emptied	42.72%	97.74%
Receiver balance stays 0 before and after	0.06%	49.65%

Measure (TRANSFER + CASH_OUT)	Legitimate	Fraud
Amount exactly 10,000,000	2,920	287

- Fraudulent amounts are larger: the median is 442,290 vs 171,021. No fraud exceeds 10 million, and 287 frauds are exactly 10,000,000.
- 97.7% of frauds empty the sender's account. The fraudster takes everything.
- In 49.6% of frauds the receiver's recorded balance stays at 0 even though money arrived, compared with 0.06% of legitimate transactions. This inconsistency is a strong fraud signal.

Possible mule accounts

Most frauds send money to a different account each time, but 43 receiver accounts received more than one fraudulent transaction (86 frauds in total). In a real bank these accounts would be referred for investigation as possible money-mule accounts. The top three by value:

Receiver account	Frauds	Total amount	First hour	Last hour
C668046170	2	10,160,088.68	269	686
C1669818195	2	8,848,239.99	446	571
C1981613973	2	7,265,501.67	72	88

9.4 Model results

Results on the 831,437-row test set (2,407 frauds):

Model	Precision	Recall	F1	AUC-PR	Caught	False alarms
Existing rule	1.000	0.002	0.003	–	4 / 2,407	0
Logistic Regression	0.043	0.909	0.083	0.549	2,189 / 2,407	48,466
Random Forest	**0.993**	**0.998**	**0.995**	**0.998**	**2,401 / 2,407**	**17**

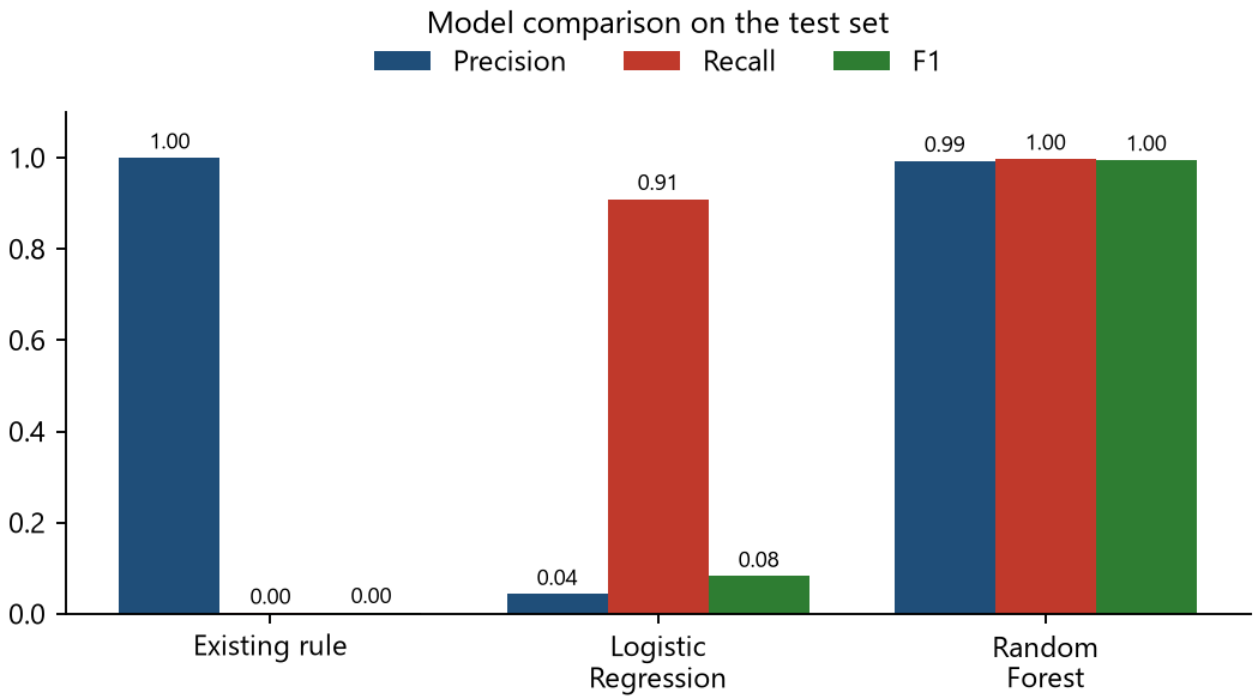


Figure 5. Precision, recall and F1 of the three approaches

- **Existing rule:** never wrong when it fires (precision 1.0), but it caught only 4 of 2,407 frauds.
- **Logistic Regression:** catches 90.9% of frauds but raises 48,466 false alarms, about 22 for every real fraud. With class weights it errs heavily on the side of flagging, and a linear boundary cannot separate the classes well.
- **Random Forest (selected):** catches 2,401 of 2,407 frauds (recall 0.9975) with only 17 false alarms (precision 0.9930). AUC-PR is 0.9978. Training took 94 seconds on 1,938,956 rows.

Confusion matrix — Random Forest

	Predicted legitimate	Predicted fraud
Actual legitimate	829,013 (TN)	17 (FP)
Actual fraud	6 (FN)	2,401 (TP)

Feature importance

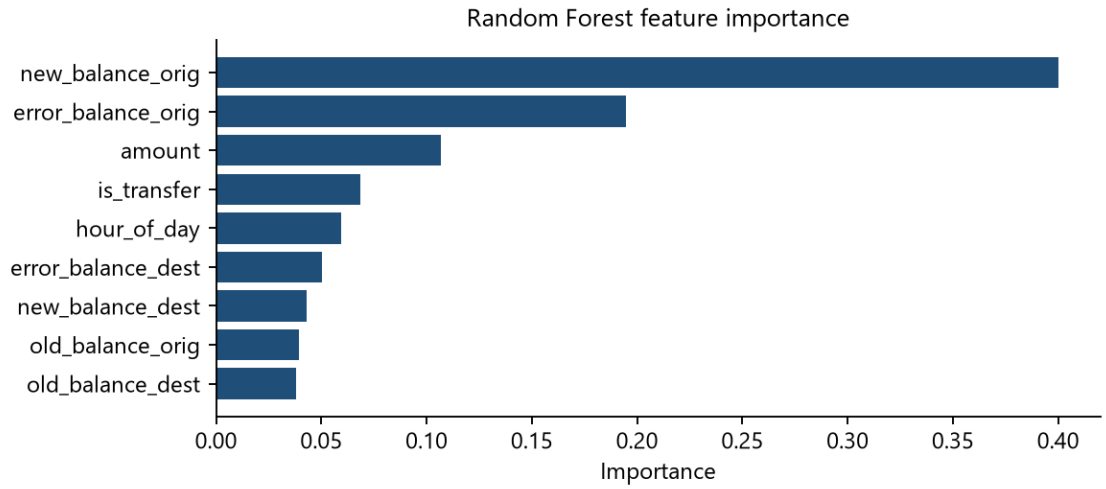


Figure 6. Which features the Random Forest relies on

The most important features are new_balance_orig (0.4001), error_balance_orig (0.1946) and amount (0.1066). In other words, the model mainly checks whether the sender's account was emptied and whether the balances add up, which matches the patterns found in the analytics.

What the model gets wrong

- All 6 missed frauds have a sender balance recorded as 0 both before and after the transaction. Without balance information the model has nothing to go on, which is the same signal it relies on most.
- The 17 false positives are legitimate transactions that look like fraud: typically the whole balance is moved out and the receiver's balance doesn't add up.

Choice of alert threshold

A transaction is flagged when its predicted fraud probability is at least 0.5. The threshold curve shows that precision and recall both stay above 0.99 for any threshold between 0.2 and 0.7. Recall only drops at 0.8 and above (0.9609 at 0.9), so the result does not depend on a carefully tuned cut-off.

10. Running the project

10.1 Requirements

- Python 3.10 or newer
- Java 17 (only needed to rerun the Spark pipeline or use the live prediction form)
- Optional: MongoDB on localhost:27017 (otherwise the dashboard uses results/*.json)
- Optional: a Kaggle API token in ~/.kaggle/ for the automatic download (otherwise download the CSV manually from Kaggle and save it as data/raw/paysim.csv)

10.2 Start the dashboard

```
git clone https://github.com/JayQuery-007/bank_transaction_analysis.git
cd bank_transaction_analysis
python -m venv venv
venv\Scripts\activate              (macOS/Linux: source venv/bin/activate)
pip install -r requirements.txt
python run.py
```

Then open http://localhost:8000 in a browser. On Windows, .\start.ps1 starts the local MongoDB first (if present in the mongodb folder) and then the dashboard.

10.3 Rebuild everything from the raw data

```
python run_pipeline.py
```

This downloads the dataset, then runs ingest, analytics and model training (about 4 minutes in total), refreshing MongoDB, results/ and models/.

10.4 Verified run

The project was run end to end for this report:

Check	Result
Fresh clone + new virtual environment + pip install -r requirements.txt	Installed without errors
Full pipeline from raw CSV (python run_pipeline.py)	Completed: ingest 42 s, analytics 41 s, model 165 s. Reproduced the same metrics.
Dashboard (python run.py) with MongoDB running	All 7 sections load; data source shows MongoDB
Dashboard without MongoDB (fresh clone)	Loads from results/*.json
Live prediction: real fraud from test set (CASH_OUT of 2,658,595.89 at 2:00, sender emptied)	Fraud probability 1.000 → Likely fraud
Live prediction: normal cash-out (18,500 of a 42,000 balance at 14:00)	Fraud probability 0.000 → Looks legitimate

10.5 Dashboard screenshots

Bank Transaction Analytics & Fraud Detection

Dataset: PaySim mobile money transactions (Kaggle) · Pipeline: Spark SQL + Spark MLlib · Serving: MongoDB + FastAPI · Data source: MongoDB

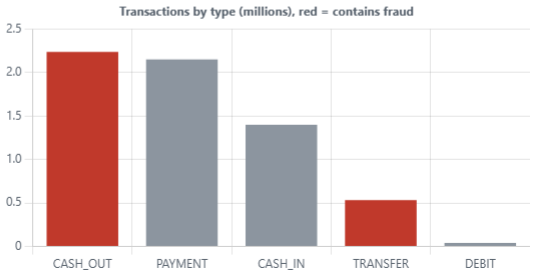
1. Dataset overview

Transactions	Total value	Period	Fraudulent transactions	Fraud value	Fraud rate
6,362,604	1144.4 bn	31 days	8,197	12.1 bn	0.129%

Raw CSV: 6,362,620 rows, 0 nulls, 0 duplicates. 16 zero-amount rows were removed. 6,353,291 distinct sender accounts and 2,722,359 distinct receiver accounts. Ingested to Parquet in 19.6s.

2. Which transaction types are affected?

Fraud occurs **only in CASH_OUT and TRANSFER** transactions. The other types have no fraud at all, so the model is trained on these two types only. The dataset's existing rule (`isFlaggedFraud`) flagged just **16** of 8,197 frauds.



Type	Transactions	Avg amount	Fraud	Fraud rate
CASH_OUT	**2,237,484**	**176,275**	**4,100**	**0.183%**
PAYMENT	2,151,495	13,058	0	0.000%
CASH_IN	1,399,284	168,920	0	0.000%
TRANSFER	**532,909**	**910,647**	**4,097**	**0.769%**
DEBIT	41,432	5,484	0	0.000%

Figure 7. Header, dataset overview and transaction types

3. When does fraud happen?

The number of frauds stays almost constant through the day (274–371 per hour slot). Legitimate activity drops sharply overnight, so the **fraud rate peaks at 22.3% around 5:00**, compared with under 0.1% during business hours.

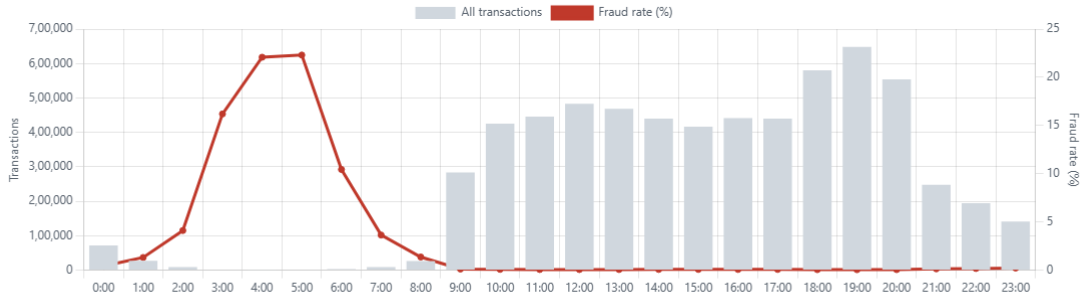
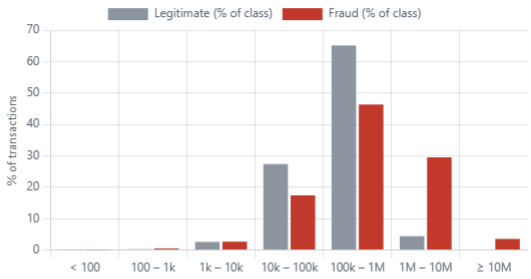


Figure 8. Time-of-day analysis

4. What do fraudulent transactions look like?

TRANSFER and CASH_OUT transactions only.

Transaction amount



Account balance behaviour

	Legitimate	Fraud
Transactions	2,762,196	8,197
Average amount	314,116	1,470,833
Median amount	171,021	442,290
Sender account emptied	42.72%	97.74%
Receiver balance stays 0	0.06%	49.65%
Amounts at the 10M cap	2,920	287

97.7% of frauds empty the sender's account (vs 42.7% of legitimate transactions), and in 49.6% of frauds the receiver's recorded balance stays at 0 even though money arrived.

Destination accounts receiving repeated fraud (possible mule accounts)

43 receiver accounts were the destination of more than one fraudulent transaction (86 frauds in total). Top 10 by value:

Receiver account	Fraud transactions	Total amount	First seen (hour)	Last seen (hour)
C668046170	2	10,160,088.68	269	686
C1669818195	2	8,848,239.99	446	571
C1981613973	2	7,265,501.67	72	88
C1366192319	2	7,061,799.65	84	344
C964377943	2	6,014,862.39	300	401
C14138104	2	4,597,314.28	415	436
C1399829166	2	3,540,009.28	212	314
C643624257	2	3,071,716.97	250	279
C1013511446	2	2,994,972.2	9	74
C1601170327	2	2,842,694.99	148	281

Figure 9. Amounts, balance behaviour and possible mule accounts

5. Fraud detection model (Spark MLlib)

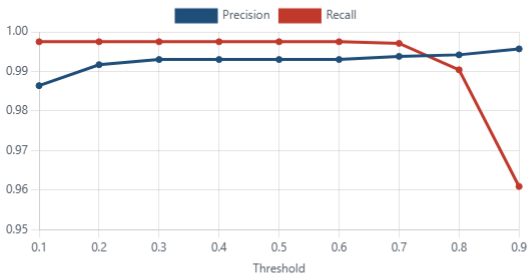
Trained on 1,938,956 TRANSFER/CASH_OUT transactions and evaluated on a held-out test set of 831,437 (2,407 frauds). Fraud is only 0.30% of this data, so accuracy is meaningless: the table reports precision, recall and area under the precision-recall curve.

Model	Precision	Recall	F1	AUC-ROC	AUC-PR	Frauds caught	False alarms	Train time
Logistic Regression	0.043	0.909	0.083	0.979	0.549	2,189 / 2,407	48,466	18.9s
Random Forest	**0.993**	**0.998**	**0.995**	**0.999**	**0.998**	**2,401 / 2,407**	**17**	**94s**
Existing rule (isFlaggedFraud)	1.000	0.002	0.003	–	–	4 / 2,407	0	–

Random Forest – confusion matrix (test set)

	Predicted legit	Predicted fraud
Actual legit	829,013	17
Actual fraud	6	2,401

Precision / recall vs alert threshold



Lowering the threshold raises more alerts. The curve is nearly flat between 0.2 and 0.7, so the default 0.5 is a reasonable operating point.

Feature importance

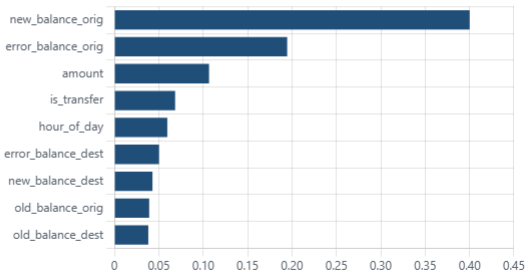


Figure 10. Model comparison, confusion matrix, feature importance and threshold curve

6. Flagged transactions (test set)

All | True positives | False positives | Missed fraud — 2,424 transactions

Day / hour	Type	Amount	Sender	Sender bal. before → after	Receiver	Receiver bal. before → after	Fraud prob.	Outcome
4 / 22:00	CASH_OUT	2,169,679.91	C395839623	2,169,680 → 0	C925758982	0 → 2,169,680	1.000	true positive
5 / 4:00	CASH_OUT	1,422,963.14	C1871604900	1,422,963 → 0	C45991423	0 → 1,422,963	1.000	true positive
3 / 7:00	CASH_OUT	854,550.85	C1410522707	854,551 → 0	C880907246	186,808 → 1,041,359	1.000	true positive
4 / 5:00	CASH_OUT	819,048.34	C48810852	819,048 → 0	C1460010933	1,104,099 → 1,923,147	1.000	true positive
3 / 23:00	CASH_OUT	1,871,553.73	C92154963	1,871,554 → 0	C1037207091	0 → 1,871,554	1.000	true positive
3 / 3:00	CASH_OUT	567,932.31	C1373739020	567,932 → 0	C1576761892	0 → 567,932	1.000	true positive
4 / 0:00	CASH_OUT	10,000,000	C333142995	10,000,000 → 0	C1990623385	841,522 → 10,841,522	1.000	true positive
2 / 4:00	CASH_OUT	3,832,058.34	C426880249	3,832,058 → 0	C1566081912	531,662 → 4,363,720	1.000	true positive
5 / 5:00	CASH_OUT	2,388,251.16	C731376303	2,388,251 → 0	C1868096235	2,624,387 → 5,012,638	1.000	true positive
2 / 6:00	CASH_OUT	236,947.25	C474899128	236,947 → 0	C116505755	70,084 → 307,032	1.000	true positive
4 / 4:00	CASH_OUT	2,066,446.27	C725597637	2,066,446 → 0	C157857444	423,087 → 2,489,533	1.000	true positive

Figure 11. Flagged transactions from the test set

7. Check a transaction

Scores a single transaction with the trained Random Forest. The first request starts Spark and takes a few seconds.

Fill with: a fraud from the test set | a normal cash-out

Type: CASH_OUT | Amount: 18500 | Hour of day: 14 | Sender balance before: 42000 | Sender balance after: 23500

Receiver balance before: 125000 | Receiver balance after: 143500 | Score transaction

Figure 12. Form for scoring a single transaction

11. Technical challenges and how they were solved

11.1 Spark cannot write files on Windows without winutils.exe

When Spark writes Parquet or saves a model on Windows, the Hadoop library underneath tries to set Unix-style file permissions and check file access by calling native helper programs (`winutils.exe`, `hadoop.dll`). Apache does not ship these for Windows, so writes fail with `HADOOP_HOME and hadoop.home.dir are unset`. The common workaround is to download these binaries from unofficial GitHub repositories.

Solution: instead of running unofficial executables, the project includes a small Hadoop FileSystem subclass (`spark_jobs/winfs/src/bda/`, about 60 lines of Java). It overrides three methods (`setPermission`, `getFileStatus`, `listStatus`) to use plain Java file operations. File permissions are meaningless on a single-user laptop. The class is compiled into `winfs.jar` (2 KB), and `common.py` enables it only when running on Windows. On Linux or macOS, and on a real cluster, Spark's normal file system is used.

11.2 PySpark Python workers crash with Python 3.13 on Windows

Some PySpark operations start separate Python worker processes, for example Python UDFs, RDD operations and PipelineModel.load(). On Windows with Python 3.12 or newer these workers crash with a socket error. Upgrading to Spark 4.0 did not fix it.

Solution: all jobs use only DataFrame, Spark SQL and MLlib operations, which run inside the JVM, plus Arrow to move results to pandas. This is also the recommended way to write efficient Spark code, because Python UDFs are slow. The API loads the saved model through the JVM directly (`spark._jvm...PipelineModel.load`).

11.3 Long file names in the saved model

Spark names saved files like part-00000-<36-character id>-c000.snappy.parquet. Cloned into a deep folder on Windows, these paths went over the 260-character limit and git clone failed.

Solution: after saving, the model's part files are renamed to short names (part-00000.parquet). Spark reads every part file in a folder regardless of its name.

11.4 Making the repository runnable without the 494 MB dataset

The raw CSV and the Parquet lake are too large for GitHub. The small outputs are committed instead: the results JSON files (under 1 MB) and the trained model (347 KB). The dashboard therefore works right after cloning, and the full pipeline can still be rerun to regenerate everything from the original data.

12. Limitations and future work

12.1 Limitations

- **Simulated data:** PaySim records balances for fraudulent transactions in a distinctive way (accounts emptied, receiver balances not updated). The model relies heavily on these balance features, which is why its scores are near-perfect. On real bank data the same approach would score lower and would need more behavioural features.
- **No customer history:** almost every sender appears only once, so features such as "spending compared with this customer's usual behaviour" or transaction velocity cannot be computed.
- **Batch only:** transactions are analysed after the fact, not as they happen.
- **Local mode:** Spark ran on one machine. The code is cluster-ready, but it was not run on a cluster.

12.2 Future work

- **Real-time scoring:** stream incoming transactions through Apache Kafka and score them with Spark Structured Streaming using the saved model, writing alerts to MongoDB.
- **Cluster deployment:** run the same jobs on a Spark cluster with the data in HDFS or cloud object storage.
- **More models:** try Gradient-Boosted Trees and an unsupervised anomaly detector, which would catch new fraud patterns without labels.
- **Graph analysis:** model sender → receiver links as a graph (for example in Neo4j) to find rings of mule accounts.

13. Viva preparation: likely questions

Q1. How does your system decide whether a transaction is fraud?

It learned from 8,197 transactions labelled as fraud (supervised learning). A Random Forest of 60 decision trees looks at the amount, the sender's and receiver's balances before and after, whether those balances add up, the transaction type and the hour. Each tree answers with a probability, the forest averages them, and if the result is 0.5 or more the transaction is flagged. In practice it flags transactions that empty the sender's whole balance, especially at night, and leave the balances inconsistent.

Q2. Where exactly is Big Data used in your project?

In every stage. 6.36M rows are ingested in parallel by Spark into a partitioned Parquet data lake. Analytics run as distributed Spark SQL with caching and partition pruning. The model is trained with Spark MLlib on all 1.94M training rows without sampling. MongoDB serves the results as a separate serving layer. The same code would run unchanged on a cluster with years of data.

Q3. Why did you use PaySim instead of real bank data?

Real transaction data with fraud labels is never published, because of customer privacy and banking regulations. PaySim was generated by a simulator calibrated on real logs from a mobile money provider, so it keeps the real statistical patterns without exposing customers. It is the standard public dataset for this problem.

Q4. Pandas can load 6 million rows. Why use Spark?

One month of one provider is already 494 MB. A real bank generates this every few days, which quickly exceeds one machine's memory. Spark processes data in parallel partitions and the same code scales to a cluster unchanged. It also gives SQL, a Parquet data lake and distributed ML in one engine.

Q5. Why convert CSV to Parquet?

Parquet is columnar and compressed: 494 MB of CSV became 315 MB, even with six extra columns. It stores the schema, reads only the columns a query needs, and supports partition pruning.

Q6. Why partition by transaction type?

Both the analysis and the model filter by type. Fraud only occurs in TRANSFER and CASH_OUT, so the model reads only those two partitions (2.77M rows) and Spark skips the other 3.6M rows without opening them.

Q7. Why MongoDB, and why not store all 6.36M rows in it?

MongoDB is the serving layer. It stores the results the dashboard needs as JSON documents, which match what the API returns. The full data stays in the Parquet lake, where Spark processes it efficiently. Copying millions of rows into the serving database would add cost without benefit.

Q8. Why didn't you use accuracy?

Fraud is 0.3% of the training data. A model that never predicts fraud would be 99.7% accurate and completely useless. Precision, recall, F1 and AUC-PR measure performance on the rare fraud class.

Q9. How did you handle class imbalance?

Logistic Regression uses class weights: each fraud row counts about 337 times, so both classes have equal total weight. Evaluation uses metrics that focus on the minority class. Random Forest handled the imbalance well without weights.

Q10. Why does Logistic Regression perform so badly?

It draws one linear boundary. The fraud signal depends on combinations, such as 'balance became 0 and the amount equals the old balance', which are non-linear. With class weights it also errs heavily towards flagging, giving about 22 false alarms per real fraud.

Q11. Isn't 0.995 F1 a sign of overfitting?

The score is measured on a 30% test set the model never saw, so it is not memorisation. The real reason is the dataset: the simulator records fraudulent balances in a distinctive way that the balance-error features capture. This is a limitation of the data, and real-world performance would be lower.

Q12. What are the error_balance features?

They compare recorded balances with what the amount implies: new sender balance + amount − old sender balance, and old receiver balance + amount − new receiver balance. For a correct transaction both are 0. Large or unusual values point to inconsistent bookkeeping, which is common in the fraud cases.

Q13. What does the threshold curve tell you?

It shows precision and recall if fraud is flagged at different probability cut-offs. Both stay above 0.99 from 0.2 to 0.7, so the model is not sensitive to the exact threshold. A bank could lower it to catch more fraud at the cost of more alerts.

Q14. How would you make this work in real time?

Publish incoming transactions to a Kafka topic, read them with Spark Structured Streaming, apply the saved Random Forest pipeline to each micro-batch, and write flagged transactions to MongoDB for the dashboard or an alerting system.

Q15. What was the hardest technical problem?

Running Spark on Windows. Writes need Hadoop's winutils.exe, which Apache doesn't provide, and PySpark's Python workers crash with Python 3.13. I wrote a small Hadoop FileSystem class in Java to replace the native calls, and kept all processing in DataFrame, SQL and MLlib operations so no Python workers are needed.

14. Project structure

Path	Purpose
`config.py`	File paths, Kaggle dataset ID, MongoDB connection settings
`run_pipeline.py`	Runs the four pipeline steps in order
`run.py`	Starts the dashboard server on http://localhost:8000
`start.ps1`	Windows: starts local MongoDB, then the dashboard
`requirements.txt`	Python dependencies
`data/download_data.py`	Downloads PaySim from Kaggle into data/raw/
`spark_jobs/common.py`	Spark session setup and publish() to MongoDB + results/
`spark_jobs/ingest.py`	Step 1: CSV → cleaned, enriched Parquet lake
`spark_jobs/analytics.py`	Step 2: Spark SQL analysis queries
`spark_jobs/train_model.py`	Step 3: MLlib training, evaluation, model saving
`spark_jobs/winfs/`	Windows file system class for Hadoop (Java source + jar)
`notebooks/eda.ipynb`	Exploratory data analysis with plots
`backend/app/main.py`	FastAPI routes
`backend/app/store.py`	Reads results from MongoDB or results/*.json
`backend/app/predict.py`	Loads the Spark model and scores single transactions
`frontend/`	Dashboard: index.html, style.css, app.js, bundled Chart.js
`models/fraud_rf/`	Trained Random Forest pipeline (Spark ML format)
`results/`	JSON copies of all result sets (also loaded into MongoDB)
`data/raw/, data/lake/`	Raw CSV and Parquet lake (created by the pipeline, not in Git)

References

1. E. A. Lopez-Rojas, A. Elmir and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," 28th European Modeling and Simulation Symposium (EMSS), 2016.

2. PaySim dataset on Kaggle: https://www.kaggle.com/datasets/ealaxi/paysim1

3. Apache Spark documentation (Spark SQL, MLlib): https://spark.apache.org/docs/3.5.3/

4. MongoDB documentation: https://www.mongodb.com/docs/

5. FastAPI documentation: https://fastapi.tiangolo.com/